



EAST WEST UNIVERSITY

PROJECT REPORT

Semester : Summer – 2026

Course Title : Data Structure

Course Code : CSE207

Section : 01

Assignment: Problem 03

Group No: 04

Submitted to:

Dr. Maheen Islam
Chairperson, Professor
Department of Computer Science & Engineering
East West University.

Submitted By:

Name	ID	Contribution
Shafkat Rahman	2025-1-60-001	Function: addEdge, isConnected, countConnected-Components. Report: Problem Definition, Program Implementation.
Abdur Rahman Moayed	2025-1-60-015	Function: createGraph, getLeafVertices, get-NoOutgoingEdge. Report: Data Structures Used, Sample Input and Output.
Sadman Showmik Anik	2025-1-60-055	Function: removeEdge, printAdjacencyList, bfs, getTotalDegree. Report: Objectives, Algorithm.

1. Problem Definition

The task assigned in this project is to implement a graph using an adjacency list and to perform a set of standard operations on it. A graph is a collection of vertices (nodes) connected by edges, and an adjacency list is a way of storing a graph where each vertex keeps a list of the other vertices it is directly connected to.

In this project, the graph is directed, meaning each edge has a specific starting vertex and ending vertex (for example, an edge from vertex 0 to vertex 1 does not automatically mean there is an edge from vertex 1 back to vertex 0). The program must let the user build such a graph from scratch, add and remove edges, print the adjacency list, traverse the graph using Breadth First Search (BFS), and analyze several properties of the graph, such as which vertices have no outgoing edges, how connected the graph is, the degree of each vertex, and how many separate connected pieces (components) the graph is made of.

Specifically, the program is required to perform the following ten operations:

1. Create a graph with a given number of vertices.
2. Add edges to the graph.
3. Remove edges from the graph.
4. Print the adjacency list representation of the graph.
5. Perform a Breadth First Search (BFS) starting from a given vertex.
6. Display all leaf vertices (end vertices) of the graph.
7. Check whether the graph is connected or not.
8. Find the total degree of all vertices.
9. Display all vertices that have no outgoing edge.
10. Count the number of connected components in the graph.

All of this is done through a simple, menu-driven console program, so the user can select any operation from a menu and try it interactively.

2. Objectives

The main learning objectives of this project are:

- To understand how a graph can be represented in memory using an adjacency list.
- To implement core graph operations such as adding and removing edges.
- To understand and implement the Breadth First Search (BFS) traversal technique.
- To analyze graph properties, including vertex degree (in-degree and out-degree), leaf vertices, connectivity, and connected components.
- To practice writing modular, readable C++ code using clearly separated functions, each handling one graph operation.
- To build a simple interactive menu-driven program that ties all these operations together.

3. Data Structures Used

To keep the program simple and beginner-friendly, this implementation avoids advanced Standard Template Library (STL) containers such as `vector` and `queue`. The graph itself is built out of linked lists, and the few supporting structures are plain, fixed-size C++ arrays. The following data structures are used:

Adjacency List (Array of Linked Lists)

The graph is stored as an array of linked lists. Every vertex u owns one singly linked list that holds exactly the vertices reachable directly from u by a single edge, and the array `adjacencyList[MAX_VERTICES]` stores the head pointer of each of those lists. A vertex with no outgoing edges simply has `NULL` as its head pointer.

Each element of a list is a node, declared as a small structure with two fields: `vertex`, which holds the number of the neighbouring vertex, and `next`, a pointer to the following node of the same list (or `NULL` at the end of the list). A node is created with `new` only when an edge is actually added, and released with `delete` as soon as that edge is removed, so the graph occupies only as much memory as the edges it currently contains. No companion counter of neighbours is needed either, because the end of a list is marked by the `NULL` pointer.

Degree Arrays

Two one-dimensional arrays, `inDegree[]` and `outDegree[]`, keep a running count of how many edges point into and out of each vertex. These are updated whenever an edge is added or removed, so the degree of any vertex can be looked up instantly instead of being recalculated every time.

Visited Array

A boolean array, `visited[]`, is used during BFS-based operations (BFS traversal, connectivity checking, and counting connected components) to keep track of which vertices have already been explored, so the program never processes the same vertex twice.

A Simple Array-Based Queue

Breadth First Search normally relies on a queue. Instead of using the STL `std::queue`, this program implements its own minimal queue using an integer array (`queueArr[]`) together with two index variables, `front` and `rear`. Adding a vertex to the queue means placing it at position `rear` and increasing `rear` by one; removing a vertex from the queue means reading the value at position `front` and increasing `front` by one. The queue is empty whenever `front` equals `rear`.

This approach keeps every data structure in the program at the level of plain arrays and simple loops, which makes the internal working of the graph fully visible and easy to trace by hand, making it a good fit for a first graph project.

4. Algorithm

4.1 Create Graph

1. Ask the user for the number of vertices, V .
2. Set every vertex's list head pointer to `NULL`, so every list starts out empty.
3. Set every vertex's in-degree and out-degree to 0.

4.2 Add Edge (u , v)

1. Check that u and v are valid vertex numbers (between 0 and $V - 1$).
2. Walk u 's linked list to see if v is already present.
3. If v is already a neighbor of u , report that the edge already exists.
4. Otherwise, create a new node holding v and attach it to the end of u 's linked list.
5. Increase `outDegree[u]` by 1 and `inDegree[v]` by 1.

4.3 Remove Edge (u , v)

1. Check that u and v are valid vertex numbers.
2. Walk u 's linked list to find the node holding v , remembering the node before it.
3. If v is not found, report that the edge does not exist.
4. Otherwise, unlink that node by making the previous node point to the node after it (or, if it was the first node, by moving the head pointer forward), and free the unlinked node.
5. Decrease `outDegree[u]` by 1 and `inDegree[v]` by 1.

4.4 Print Adjacency List

1. For every vertex from 0 to $V - 1$, print its number, then walk its linked list and print each neighbour it holds.
2. If a vertex has no neighbors, print an empty list for it.

4.5 Breadth First Search (BFS)

1. Mark the starting vertex as visited and place it in the queue.
2. While the queue is not empty, remove a vertex from the front of the queue and record it as visited in the traversal order.
3. For each neighbor of that vertex, if it has not been visited yet, mark it visited and place it at the back of the queue.
4. Repeat until the queue becomes empty, then print the full traversal order.

4.6 Leaf Vertices / Vertices with No Outgoing Edge

1. For every vertex, check whether its list head pointer is NULL.
2. Collect and print all vertices whose list is empty.

4.7 Check Connectivity

1. Starting from vertex 0, run a BFS and count how many vertices get visited.
2. If the number of visited vertices equals the total number of vertices, the graph is connected; otherwise, it is not.

4.8 Total Degree of All Vertices

1. For every vertex, add its `inDegree` and `outDegree` together to get its total degree.

2. Print each vertex's in-degree, out-degree, and total degree, and keep a running sum of all total degrees.

4.9 Count Connected Components

1. Mark all vertices as unvisited.
2. Go through each vertex in order; if it has not been visited, start a new BFS from it: this discovers one full connected component.
3. Sort and print the vertices found in that component, then increase the component counter by one.
4. Repeat until every vertex has been visited, then print the total number of components found.

5. Program Implementation

The program is written in plain, procedural C++: there is no class and no constructor. The graph itself is simply a set of global variables (the array of list head pointers `adjacencyList`, the node structure `Node` that those lists are made of, the arrays `inDegree` and `outDegree`, and the flag `graphExists`), and every operation is a plain function that reads and updates that same global data directly. A menu-driven `main()` function ties everything together by calling the right function for whichever option the user chooses. A short description of each function follows.

- `createGraph(numVertices)`: Sets up a new, empty graph with the given number of vertices. It first frees the nodes of any graph that was built earlier, then sets every head pointer to `NULL` and every in-degree and out-degree to zero, and marks that a graph now exists. This plays the role a constructor would play in an object-oriented version.
- `addEdge(u, v)`: Adds a directed edge from vertex u to vertex v by allocating a new node and linking it at the end of u 's list, after checking that the vertices are valid and the edge does not already exist.
- `removeEdge(u, v)`: Removes the directed edge from u to v if it exists by unlinking the matching node from u 's list and freeing it, then updates the degree counts accordingly.
- `printAdjacencyList()`: Walks and displays the linked list of every vertex in the graph.
- `bfs(startVertex)`: Performs a Breadth First Search starting from the given vertex and prints the order in which vertices are visited.
- `getLeafVertices()`: Finds and prints every vertex that currently has no outgoing edges.
- `isConnected()`: Uses BFS from vertex 0 to determine and report whether the whole graph is reachable from a single starting point.
- `getTotalDegree()`: Prints the in-degree, out-degree, and total degree of every vertex, along with the sum of all degrees.
- `getNoOutgoingEdge()`: Prints every vertex that has no outgoing edge (the same condition as a leaf vertex in this program).
- `countConnectedComponents()`: Repeatedly runs BFS from any not-yet-visited vertex to

find and print every connected component in the graph, along with the total number of components.

- `displayMenu()`: Prints the list of available operations for the user to choose from.
- `main()`: Repeatedly displays the menu, reads the user's choice, validates it, and calls the matching function (`createGraph`, `addEdge`, `bfs`, and so on) until the user chooses to exit.

All of these functions live together in a single file so the whole program is easy to read from top to bottom, with the global graph data and its functions defined first and the menu-driven `main()` program below them.

6. Sample Input and Output

The following run demonstrates the program in action. A graph with 6 vertices (numbered 0 to 5) is created, and five directed edges are added: $0 \rightarrow 1$, $0 \rightarrow 2$, $1 \rightarrow 3$, $2 \rightarrow 3$, and $3 \rightarrow 4$. Vertex 5 is deliberately left with no edges, so the graph has one disconnected vertex.

Step 1: Create the graph and add edges

```
Enter your choice (0-10): 1
Enter number of vertices: 6
Graph created with 6 vertices

Enter your choice (0-10): 2
Enter source vertex: 0
Enter destination vertex: 1
Edge added: 0 -> 1

Enter your choice (0-10): 2
Enter source vertex: 0
Enter destination vertex: 2
Edge added: 0 -> 2

Enter your choice (0-10): 2
Enter source vertex: 1
Enter destination vertex: 3
Edge added: 1 -> 3

Enter your choice (0-10): 2
Enter source vertex: 2
Enter destination vertex: 3
Edge added: 2 -> 3

Enter your choice (0-10): 2
Enter source vertex: 3
Enter destination vertex: 4
Edge added: 3 -> 4
```

Step 2: Print the adjacency list (option 4)

```
Adjacency List Representation:
-----
Vertex 0:  1 2
Vertex 1:  3
Vertex 2:  3
Vertex 3:  4
Vertex 4:  []
Vertex 5:  []
-----
```

Step 3: BFS from vertex 0 (option 5)

```
Enter starting vertex for BFS: 0

BFS starting from vertex 0:
-----
Visiting:  0
Visiting:  1
Visiting:  2
Visiting:  3
Visiting:  4
BFS traversal order:  0 1 2 3 4
-----
```

Step 4: Leaf vertices (option 6)

```
Leaf Vertices (no outgoing edges):
-----
Leaf vertices:  4 5
-----
```

Step 5: Connectivity check (option 7)

```
Connectivity Check:
-----
The graph is NOT CONNECTED
-----
```

The graph is reported as not connected because vertex 5 has no edges linking it to any other vertex.

Step 6: Total degree of all vertices (option 8)

```
Total Degree of All Vertices:
-----
Vertex 0:  in_degree=0, out_degree=2, total_degree=2
Vertex 1:  in_degree=1, out_degree=1, total_degree=2
Vertex 2:  in_degree=1, out_degree=1, total_degree=2
Vertex 3:  in_degree=2, out_degree=1, total_degree=3
Vertex 4:  in_degree=1, out_degree=0, total_degree=1
Vertex 5:  in_degree=0, out_degree=0, total_degree=0
Sum of all degrees:  10
-----
```

Step 7: Vertices with no outgoing edge (option 9)

```
Vertices with No Outgoing Edge:
-----
Vertices with no outgoing edges:  4 5
-----
```

Step 8: Connected components (option 10)

```
Connected Components:
-----
Component 1:  0 1 2 3 4
Component 2:  5
Number of connected components:  2
-----
```

As expected, the program finds two components: the main group $\{0, 1, 2, 3, 4\}$ and the isolated vertex $\{5\}$.

Step 9: Remove an edge and print the adjacency list again (options 3 and 4)

```
Enter your choice (0-10):  3
Enter source vertex:  0
Enter destination vertex:  1
Edge removed:  0 -> 1

Enter your choice (0-10):  4

Adjacency List Representation:
-----
Vertex 0:  2
Vertex 1:  3
Vertex 2:  3
Vertex 3:  4
Vertex 4:  []
Vertex 5:  []
-----
```